

AI-Native Software Development

The paradigm shift from typing code to architecting systems.

Before (Weeks/Days)

- **Idea to plan:** Long documents, meetings
- **Writing code:** Type every line, search Stack Overflow
- **Debugging:** Read error, guess, search
- **Testing:** Write tests by hand, often skipped
- **Docs:** Written last, often never

Now (Hours/Minutes)

- **Idea to plan:** Describe it to AI, get a draft plan in minutes
- **Writing code:** AI writes a first draft, you review and fix
- **Debugging:** Paste error, AI explains and suggests a fix
- **Testing:** AI generates tests, you check they make sense
- **Docs:** AI drafts docs from the code

Thinking replaces typing. The slow parts got faster, moving your time to reviewing and designing.

Demystifying Generative AI

AI that creates new content (text, code, images, audio, video)
by learning patterns from massive datasets.

Traditional AI (The Classifier)

Input: A photo

Classify

Output: Cat (Picks a fixed label)

Generative AI (The Creator)

Input: Write a login page in React

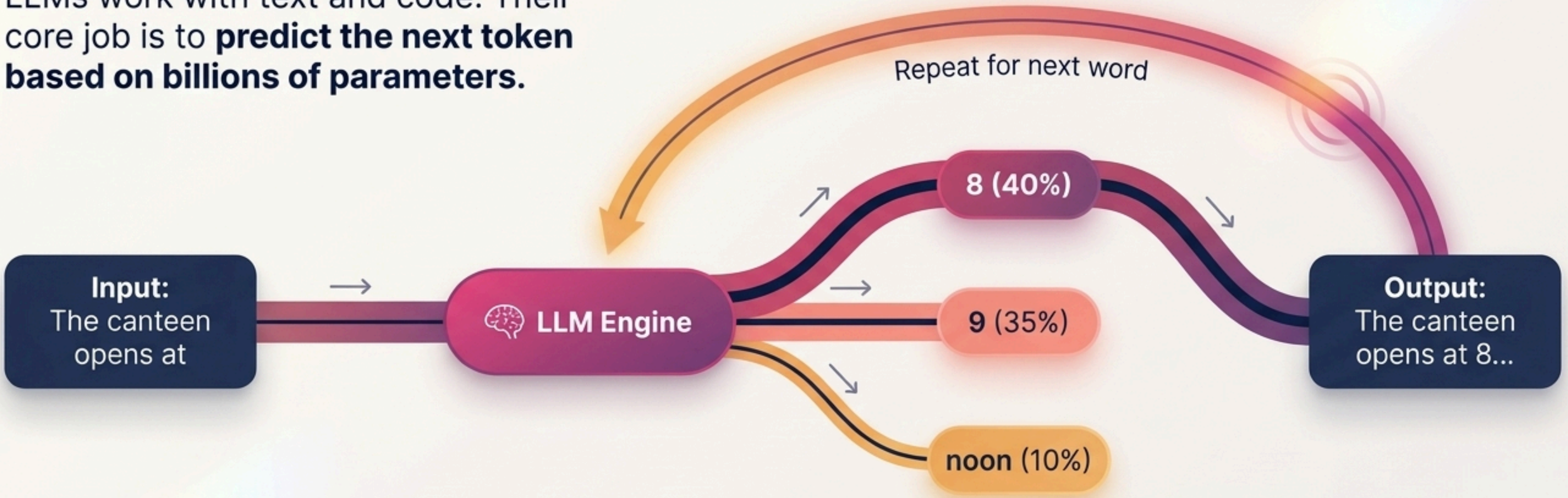
Generate

Output: 40 lines of net-new code
(Creates something that did not exist before)

ChatGPT | Gemini | Claude | Midjourney | GitHub Copilot

The Core Engine: Large Language Models

LLMs work with text and code. Their core job is to **predict the next token based on billions of parameters.**



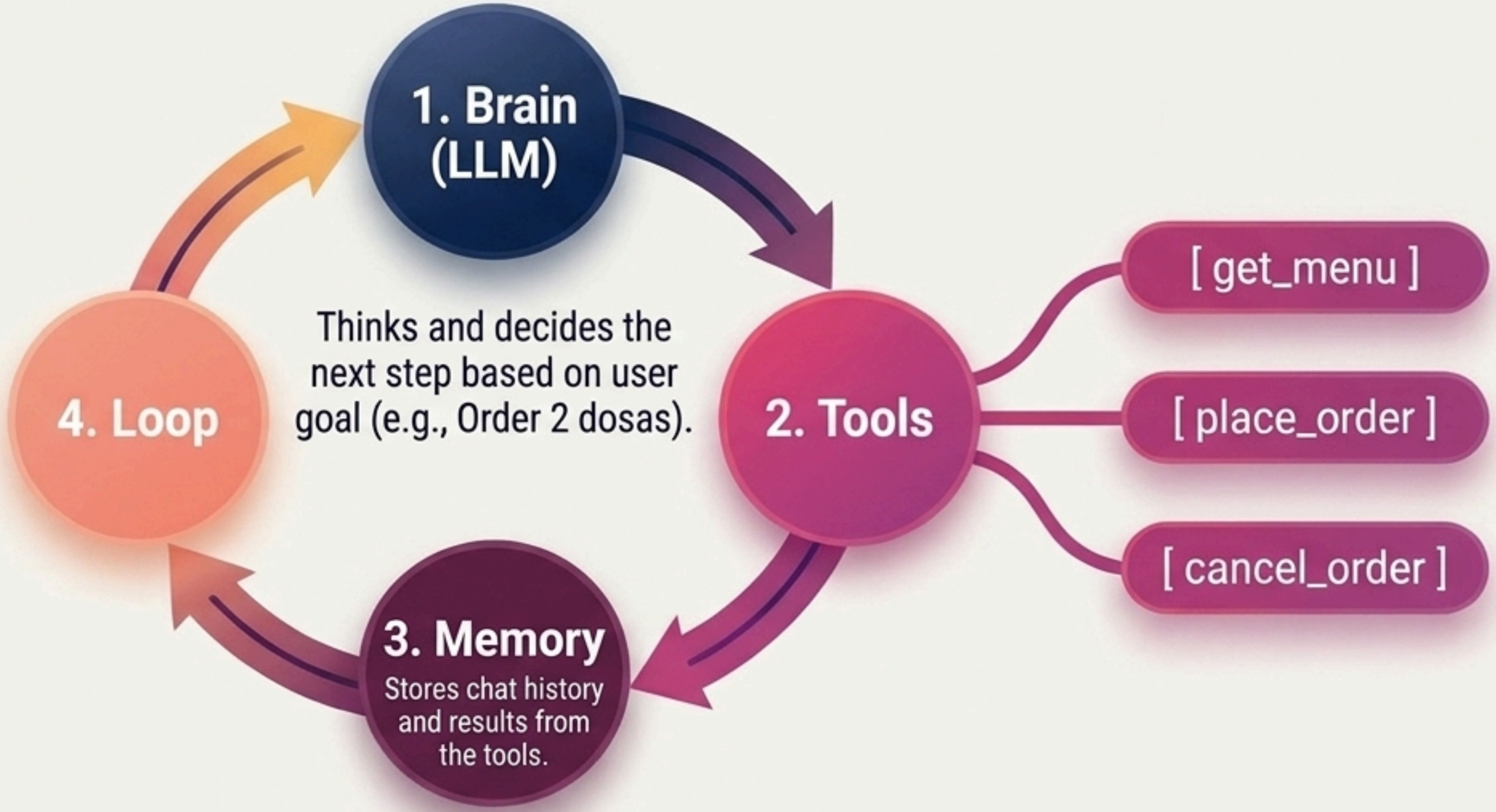
Gemini GPT Claude Llama

Core Insight: LLMs sound confident because they are always choosing a mathematically likely next word, not fact-checking. This is why they hallucinate.

The AI Evolution Matrix

GenAI Chat [TALKS]	Coding Assistant [HELPS]	AI Agent [DOES]
What it does: Answers your question.	**What it does:** Suggests code as you type.	**What it does:** Plans and takes actions to finish a task.
Interaction: You ask -> It answers.	**Interaction:** You type -> It suggests -> You accept.	**Interaction:** You give a goal -> It plans -> Acts -> Checks -> Repeats.
Can it act?: No, only replies.	**Can it act?:** Edits code you accept.	**Can it act?:** Yes (runs tools, edits files, calls APIs).
Who drives?: You.	**Who drives?:** You.	**Who drives?:** AI, with you watching.
Examples: ChatGPT, Gemini app.	**Examples:** Copilot autocomplete.	**Examples:** Antigravity agent, CampusBite assistant.

Anatomy of an AI Agent



Is the goal done?
If No: Loop back to Brain.
If Yes: Deliver final answer.

Insight: Without tools, an LLM can only talk. Tools empower the LLM to change the real world (e.g., updating a database).

The AI-Assisted Lifecycle



At every stage: AI drafts -> Human reviews. The human review arrow never goes away

Trust vs. Caution

Trust AI for speed, never for correctness.

Does Well - Trust for Speed

- Boilerplate and repeated code
- Explaining unfamiliar code
- Writing first drafts fast
- Converting between languages
- Generating test cases
- Fixing common errors

Fails or Struggles - Verify for Correctness

- Inventing APIs or packages that do not exist (Hallucinations)
- Security (e.g., leaking secret keys, skipping checks)
- Large apps where it cannot see all files
- Latest library versions (Outdated knowledge)
- Business rules it was never told
- Knowing when it is wrong (Sounds equally confident)

Real-World Danger: AI writing a Supabase query with the secret key in the browser code. It works, but anyone can steal your key.

Your Evolution: From Typist to Architect

Old Role

Typist: Writes all the code manually.



New Role

Architect: Decides what to build, how it fits together, and the overall design.

Prompt Writer: Gives clear, precise instructions to the AI.

Reviewer: Reads and checks every single line of generated code.

Owner: Ultimately responsible for the final result.

AI makes a good developer faster. It does not make a non-developer a developer. To review code well, you must understand the basics.